

CASE STUDY DOCUMENT

Online Food Delivery Platform

QuickBite*Order Smarter. Eat Better. Delivered Faster.*

Version 1.0 | 2026

1. Synopsis

QuickBite is a full-stack Online Food Delivery Platform inspired by Zomato and Swiggy. It connects customers, restaurants, and delivery agents on a single unified platform. Customers can discover nearby restaurants, browse category-wise menus, build a cart, place orders, track deliveries in real time, and rate their experience. Restaurant owners can register their establishments, manage menus, toggle item availability, and track incoming orders and earnings. Delivery agents can register, view assigned orders, update live location, and mark deliveries complete.

The platform supports four primary roles: Guests (unauthenticated visitors), Customers, Restaurant Owners, and Delivery Agents — each with a purpose-built dashboard — along with a Platform Administrator who oversees the entire ecosystem. QuickBite is architected as a microservices system modelled on the EShoppingZone reference pattern, with independent services for authentication/user management, restaurant profiles, menu management, cart management, order lifecycle, payment/wallet, delivery agent management, reviews/ratings, and notifications — all integrated through a Spring MVC website controller layer.

2. Detailed Requirements

2.1 General Requirements

- Guests can browse restaurants, view menus, and search without logging in.
- Users must register or log in (email/password or OAuth) to place orders or manage profiles.
- Role-based access control: Customer, Restaurant Owner, Delivery Agent, and Admin.
- All users can update their profile, contact information, and password.
- In-app, email, and SMS notifications are dispatched for all key order lifecycle events.
- Admin panel provides full platform management, restaurant approval, agent verification, and analytics.

2.2 Customer Requirements

- Register and log in via email/password or Google/GitHub OAuth.
- Browse nearby restaurants filtered by location, cuisine, rating, delivery time, or cost-for-two.
- View detailed restaurant profiles including menu categories, item descriptions, prices, and ratings.
- Add items to cart from a single restaurant; prompted to clear cart when switching restaurants.
- Update item quantities in cart; remove items; apply promo codes for discounts.
- Place orders with Cash on Delivery (COD) or online payment (wallet, card, UPI).
- Track order status in real-time: Placed → Confirmed → Preparing → Picked Up → Delivered.
- View complete order history; reorder from any past order in one click.
- Cancel orders within an allowed time window before preparation begins.
- Rate restaurants (food quality) and delivery agents (delivery experience) after order completion.

- Add money to wallet; view current balance and full transaction statement history.
- Receive notifications for: order confirmation, preparation started, picked up, delivered, payment receipt.

2.3 Restaurant Owner Requirements

- Register a restaurant with name, cuisine, address, GPS coordinates, delivery radius, and operating hours.
- Await admin approval before becoming visible to customers on the platform.
- Manage menu: create and organize menu categories; add, edit, and delete menu items.
- Toggle item availability (in-stock / out-of-stock) in real time.
- View and accept incoming orders; update order status through the pipeline.
- Set estimated delivery time and minimum order amount thresholds.
- View earnings analytics: daily/weekly/monthly revenue, top-selling items, peak hours.
- Receive in-app and audio notifications for new incoming orders.
- View all reviews and ratings from customers; flag inappropriate reviews for moderation.

2.4 Delivery Agent Requirements

- Register as a delivery agent with personal details, vehicle type, and vehicle registration number.
- Await admin verification before being eligible for order assignments.
- Toggle availability (online/offline) to start or stop receiving orders.
- View assigned orders with restaurant pickup location and customer delivery address.
- Update live GPS coordinates at configurable intervals for real-time tracking by customers.
- Mark orders as picked up from restaurant and delivered to customer.
- View earnings summary and customer delivery ratings.

2.5 Admin Requirements

- View and manage all user accounts: customers, restaurant owners, and delivery agents — suspend, reactivate, or delete.
- Review and approve new restaurant registrations; reject with feedback if documents are incomplete.
- Verify delivery agent identity and vehicle documents before activating their account.
- View all orders platform-wide with full lifecycle status and payment details.
- View all payment transactions; initiate refunds where applicable.
- View and moderate all customer reviews; remove fraudulent or inappropriate content.
- Access platform analytics: total orders, revenue, most-ordered cuisines, peak delivery zones.
- Send platform-wide notifications (promotions, maintenance alerts) to all or targeted users.
- Generate revenue reports for merchant payouts and platform commission reconciliation.

2.6 Payment & Wallet Requirements

- Customers can pay via COD, debit/credit card, UPI, or wallet balance.
- Wallet supports deposit (add money) and debit (payment) operations; each generates a statement record.
- Wallet balance is validated before wallet payment is accepted to prevent negative balance.
- Refunds are credited back to the original payment mode or to the wallet within 3-5 business days.
- Payment records store amount, mode, transaction ID, status, and timestamps for full audit.

2.7 Notification Requirements

- Notifications dispatched for: order placed, order confirmed by restaurant, food being prepared, picked up by agent, delivered.
- Each notification carries recipientId, type, relatedId (orderId), and a deep-link URL.
- Multi-channel: in-app notification centre, email (JavaMailSender), and SMS (Twilio/AWS SNS).
- Restaurant owners receive audio + in-app alert for every new incoming order.
- Unread badge count displayed in real time in the navigation bar.
- Admin can broadcast platform-wide promotional or informational notifications.

3. Use Case Diagram

The diagram below captures all actors and use cases within the QuickBite platform — spanning authentication, restaurant discovery, cart and ordering, real-time delivery tracking, payment, review, notifications, and full administrative management.

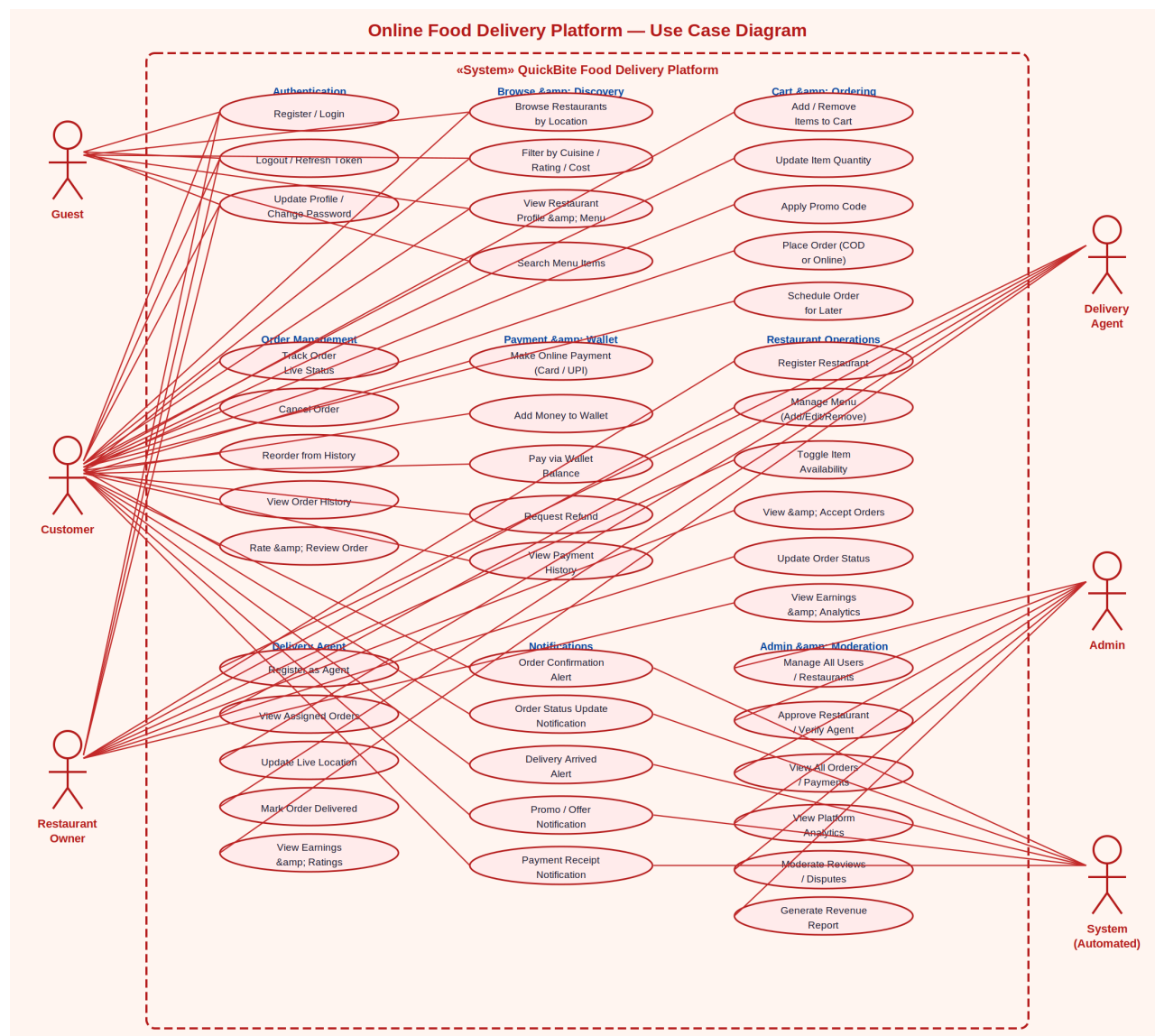


Figure 1: QuickBite Food Delivery Platform — Complete Use Case Diagram

3.1 Actors

Actor	Description
Guest	Unauthenticated visitor who can browse restaurants and view menus
Customer	Registered user who orders food, pays, tracks delivery, and submits reviews
Restaurant Owner	Merchant who registers a restaurant, manages the menu, and processes incoming orders
Delivery Agent	Registered rider who accepts delivery assignments, updates location, and marks orders delivered
Admin	Platform administrator managing users, restaurants, agents, payments, and analytics
System (Automated)	Automated component handling notification dispatch, order timeout, and refund processing

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register / Login	Guest, Customer, Owner, Agent	Create account or authenticate via email or OAuth
Update Profile	Customer, Owner, Agent	Change name, phone, profile picture, or password
Browse Restaurants	Guest, Customer	View nearby restaurants filtered by location and preferences
Filter by Cuisine / Rating	Guest, Customer	Narrow restaurant list by cuisine type, rating, cost, or delivery time
View Restaurant & Menu	Guest, Customer	See restaurant details, menu categories, and item descriptions with prices
Search Menu Items	Guest, Customer	Find specific dishes across the restaurant's full menu
Add Items to Cart	Customer	Select menu items and quantities into a session cart
Update Item Quantity	Customer	Increase or decrease quantity of any item in the cart
Apply Promo Code	Customer	Enter a discount code to reduce the order total
Place Order (COD)	Customer	Confirm order with cash on delivery payment mode
Place Order (Online)	Customer	Pay via card, UPI, or wallet at checkout

Use Case	Actor(s)	Description
Track Order Live	Customer	View real-time order status updates and delivery agent location
Cancel Order	Customer	Cancel before restaurant begins preparation; trigger refund if applicable
View Order History	Customer	Browse all past orders with status, items, and total amount
Reorder	Customer	Recreate a past order's cart with a single click
Rate & Review Order	Customer	Submit food rating and delivery experience rating after completion
Add Money to Wallet	Customer	Top up wallet balance via card or UPI for future orders
Pay via Wallet	Customer	Debit wallet balance to pay for an order at checkout
Request Refund	Customer	Submit refund request for cancelled or undelivered orders
View Payment History	Customer	See all payment transactions and wallet statement records
Register Restaurant	Restaurant Owner	Submit restaurant details for admin approval and onboarding
Manage Menu	Restaurant Owner	Add/edit/delete categories and menu items; set prices and images
Toggle Item Availability	Restaurant Owner	Mark items as in-stock or out-of-stock in real time
View & Accept Orders	Restaurant Owner	See incoming order queue; confirm and begin preparation
Update Order Status	Restaurant Owner	Progress order through Confirmed → Preparing → Ready for Pickup stages
View Earnings & Analytics	Restaurant Owner	Access revenue by period, top items, and order trends
Register as Agent	Delivery Agent	Submit personal and vehicle details for admin verification
View Assigned Orders	Delivery Agent	See pending pickup and active delivery assignments
Update Live Location	Delivery Agent	Send GPS coordinates at intervals for customer live tracking
Mark Order Delivered	Delivery Agent	Complete the delivery and release the slot for the next order
View Agent Earnings	Delivery Agent	Check delivery fees earned and customer ratings received

Use Case	Actor(s)	Description
Approve Restaurant / Agent	Admin	Review and approve or reject new registrations and verifications
Manage All Users	Admin	View, suspend, reactivate, or delete any platform user
View All Orders / Payments	Admin	Monitor every order and transaction across the platform
View Platform Analytics	Admin	Access total GMV, order volumes, peak zones, and top cuisines
Moderate Reviews	Admin	Remove fraudulent, offensive, or manipulated customer reviews
Generate Revenue Report	Admin	Export merchant payout and commission reconciliation data
Send Platform Notification	Admin	Broadcast promotions or alerts to all or targeted users
Send Order Notifications	System	Auto-dispatch order status alerts via app, email, and SMS
Process Refund	System	Automatically credit refund on successful cancellation validation

4. Class Diagrams — All Services

QuickBite follows the same layered microservices architecture as the EShoppingZone reference system. Each service comprises five layers: Entity/POJO (domain model), Repository Interface (data access), Service Interface (business contract), ServiceImpl (business logic), and REST Resource (API exposure). The eleven class diagrams below detail every service in the platform.

4.1 Auth / User-Service

The Auth/User-Service is the security gateway for the entire platform. It manages user registration, login, JWT token lifecycle, OAuth2 provider integration (Google/GitHub), profile management, password changes, and account deactivation. Every protected endpoint on the platform verifies the JWT through this service, and user identity resolution for all roles (Customer, Restaurant Owner, Delivery Agent, Admin) is centralised here.

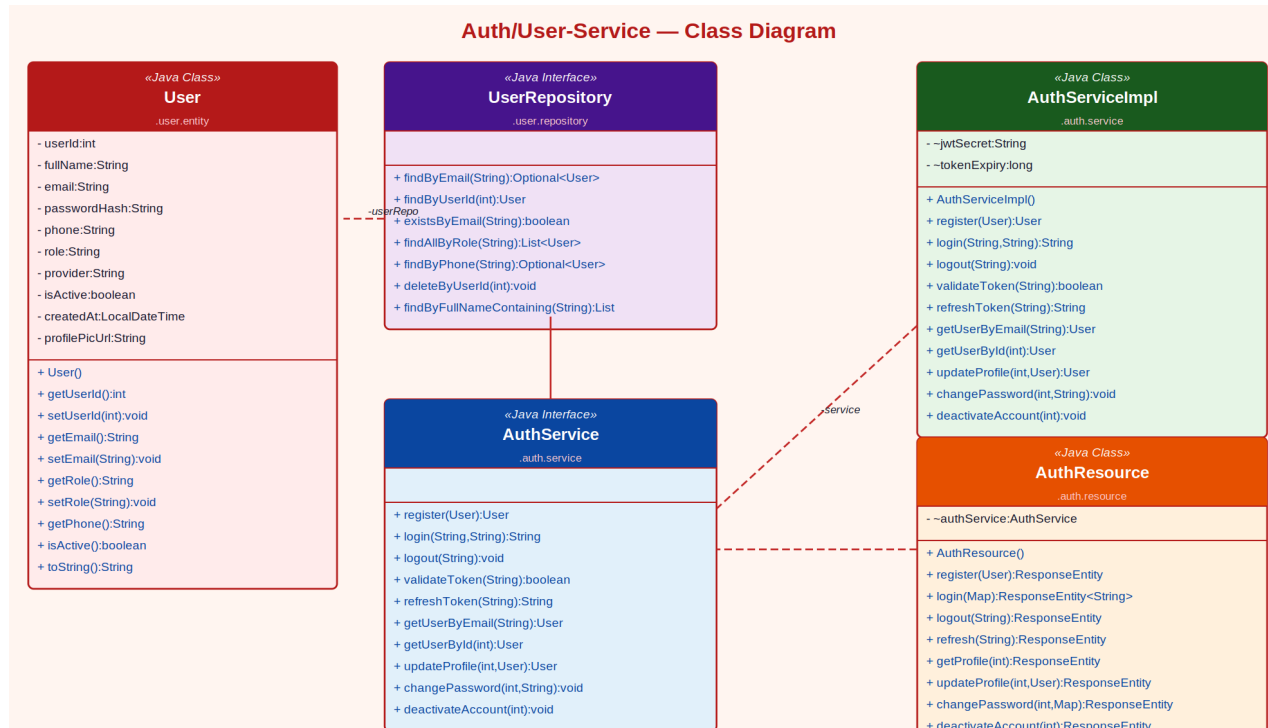


Figure 2: Auth/User-Service — Class Diagram

Component	Type	Responsibility
User	Java Class (Entity)	Stores userId, fullName, email, passwordHash, phone, role (CUSTOMER/OWNER/AGENT/ADMIN), provider (OAuth), isActive, createdAt, profilePicUrl
UserRepository	Java Interface	findByEmail(), findById(), existsByEmail(), findAllByRole(), findByPhone(), findByFullNameContaining(), deleteById()
AuthServiceImpl	Java Class	Implements register, login, logout, JWT generation/validation, OAuth handling, profile update, password change, account deactivation
AuthService	Java Interface	Declares register(), login(), logout(), validateToken(), refreshToken(), getUserByEmail(), getUserById(), updateProfile(), changePassword(), deactivateAccount()

Component	Type	Responsibility
AuthResource	Java Class (REST)	Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile (GET/PUT), /auth/password, /auth/deactivate

4.2 Restaurant-Service

The Restaurant-Service manages the profiles of all registered restaurants. It stores geolocation (latitude/longitude) for proximity-based discovery, cuisine tags, delivery radius, minimum order amount, and estimated delivery time. Admin approval is required before a restaurant is visible to customers. The service aggregates average food ratings updated by the Review-Service after each submission.

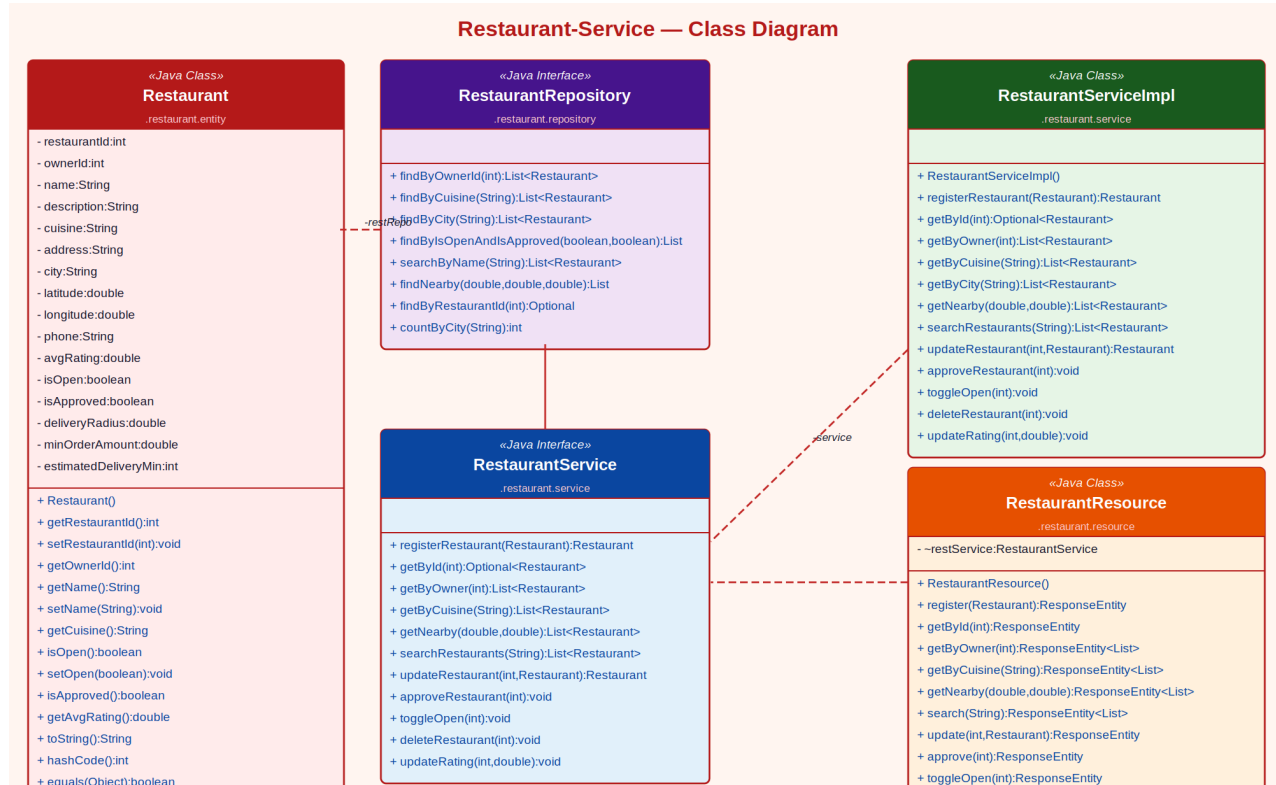


Figure 3: Restaurant-Service — Class Diagram

Component	Type	Key Attributes / Methods
Restaurant	Java Class (Entity)	restaurantId, ownerId, name, description, cuisine, address, city, latitude, longitude, phone, avgRating, isOpen, isApproved, deliveryRadius, minOrderAmount, estimatedDeliveryMin
RestaurantRepository	Java Interface	findByOwnerId(), findByCuisine(), findByCity(), findByIsOpenAndIsApproved(), searchByName(), findNearby(), countByCity()
RestaurantServiceImpl	Java Class	registerRestaurant(), getById(), getByOwner(), getByCuisine(), getByCity(), getNearby(), searchRestaurants(), updateRestaurant(), approveRestaurant(), toggleOpen(), deleteRestaurant(), updateRating()

Component	Type	Key Attributes / Methods
RestaurantService	Java Interface	Declares all restaurant CRUD, search, geo-proximity, approval, open-toggle, and rating operations
RestaurantResource	Java Class (REST)	Exposes /restaurants endpoints: POST (register), GET (by id/owner/cuisine/city/nearby/search), PUT (update/approve/toggleOpen), DELETE

4.3 Menu-Service

The Menu-Service manages the full menu structure of each restaurant. Menus are organised into MenuCategories (e.g., Starters, Main Course, Desserts), each containing a list of MenuItems. Items carry nutritional metadata (calories, tags), a discounted price field, and an availability toggle so owners can instantly mark items as sold out without deleting them. The service supports veg/non-veg filtering and keyword search across menu items.

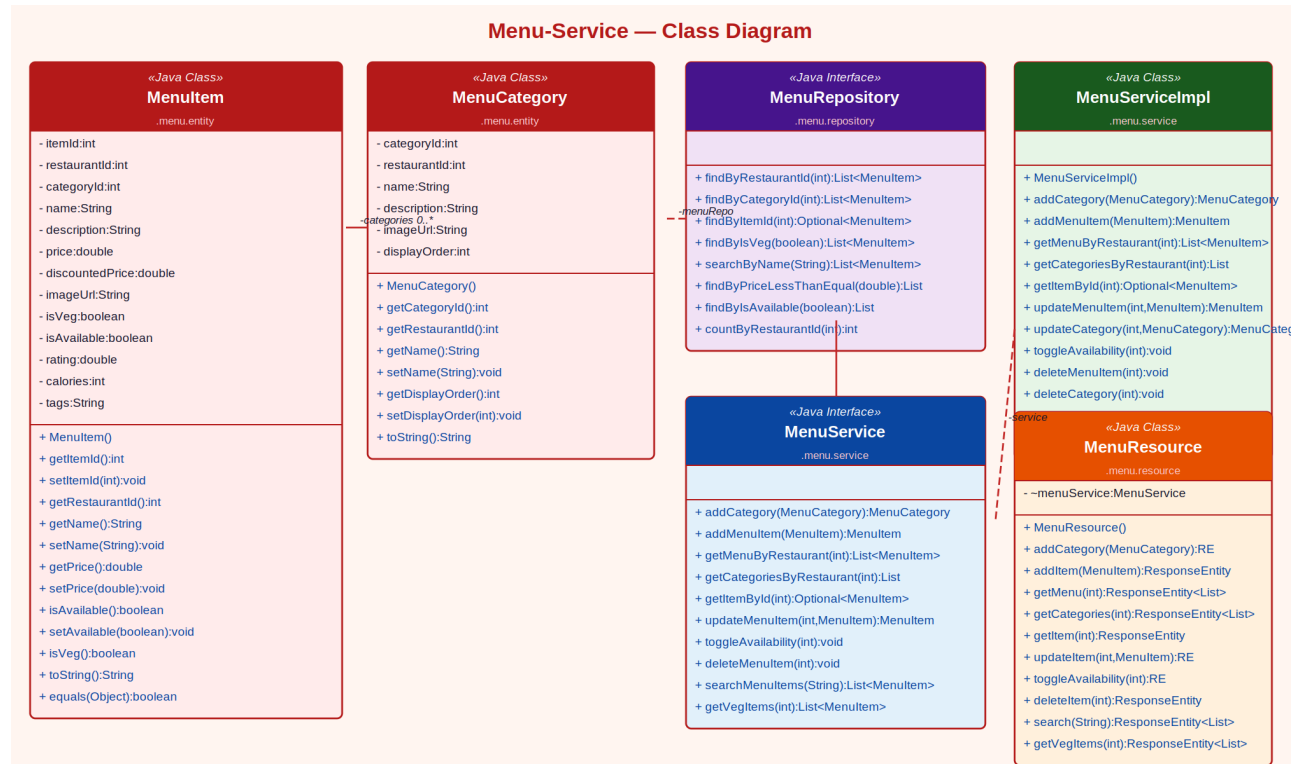


Figure 4: Menu-Service — Class Diagram

Component	Type	Key Attributes / Methods
Menuitem	Java Class (Entity)	itemId, restaurantId, categoryId, name, description, price, discountedPrice, imageUrl, isVeg, isAvailable, rating, calories, tags
MenuCategory	Java Class (Entity)	categoryId, restaurantId, name, description, imageUrl, displayOrder — groups MenuItems into sections
MenuRepository	Java Interface	findByRestaurantId(), findByCategoryId(), findById(), findByIsVeg(), searchByName(), findByPriceLessThanEqual(), findByIsAvailable(), countByRestaurantId()
MenuServiceImpl	Java Class	addCategory(), addMenuItem(), getMenuByRestaurant(), getCategoriesByRestaurant(), getItemById(),

Component	Type	Key Attributes / Methods
		updateMenuItem(), updateCategory(), toggleAvailability(), deleteMenuItem(), deleteCategory(), searchMenuItems(), getVegItems()
MenuService	Java Interface	Declares all category and item CRUD, availability toggle, search, and veg-filter operations
MenuResource	Java Class (REST)	Exposes /menu endpoints: POST (category/item), GET (by restaurant/category/id), PUT (update/toggleAvailability), DELETE, GET (search/vegItems)

4.4 Cart-Service

The Cart-Service manages a single active cart per customer. Each Cart is associated with one restaurant (to enforce single-restaurant ordering) and contains a list of CartItems, each capturing the menu item reference, price snapshot, quantity, and customisation notes. The service validates that all items belong to the cart's restaurant, computes the total, and supports promo code application via inter-service validation.

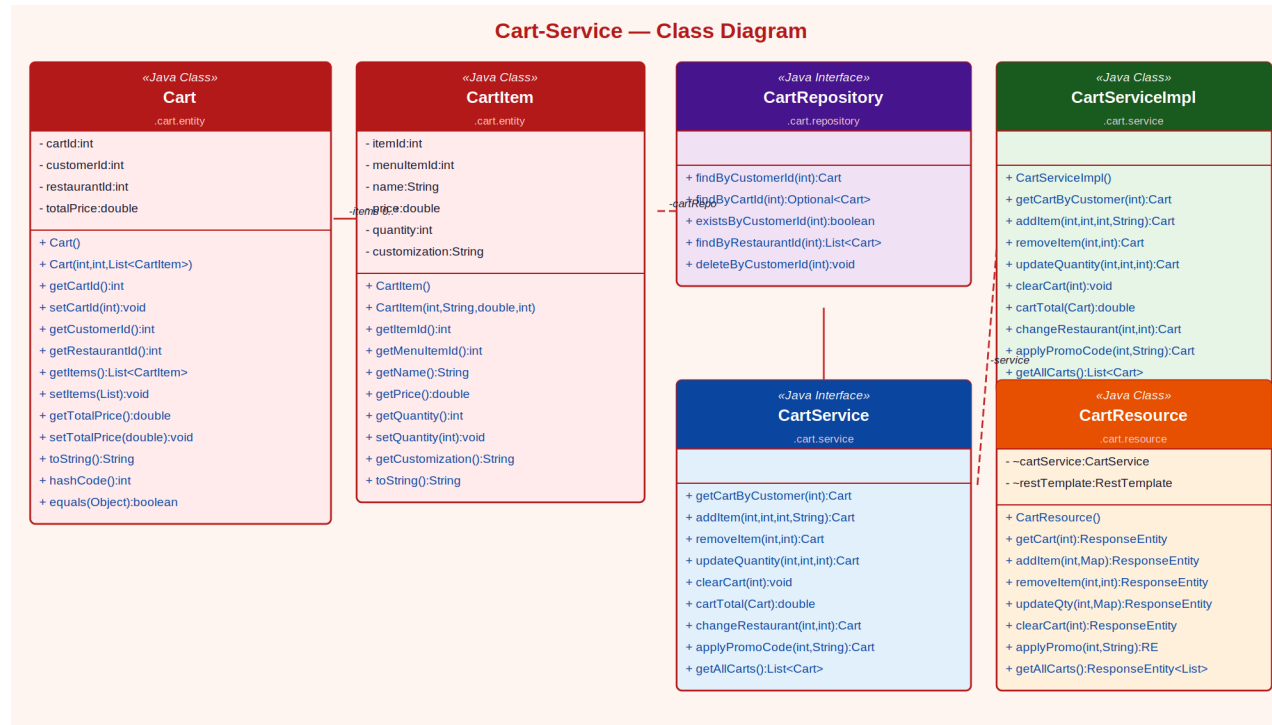


Figure 5: Cart-Service — Class Diagram

Component	Type	Key Attributes / Methods
Cart	Java Class (Entity)	cartId, customerId, restaurantId, totalPrice; contains 0..* CartItems
CartItem	Java Class (Entity)	itemId, menuItemId, name, price, quantity, customization — price snapshot at time of add
CartRepository	Java Interface	findByCustomerId(), findByCartId(), existsByCustomerId(), findByRestaurantId(), deleteByCustomerId()
CartServiceImpl	Java Class	getCartByCustomer(), addItem(), removeItem(), updateQuantity(), clearCart(), cartTotal(), changeRestaurant(), applyPromoCode(), getAllCarts()

Component	Type	Key Attributes / Methods
CartService	Java Interface	Declares all cart retrieval, item management, promo application, and restaurant-switch operations
CartResource	Java Class (REST)	Exposes /cart endpoints: GET (by customer), POST (addItem), DELETE (removeItem/clear), PUT (updateQty), POST (applyPromo), GET (allCarts)

4.5 Order-Service

The Order-Service is the central orchestration service of the platform. It converts a confirmed cart into an Order, records OrderItems as a snapshot, and manages the complete order lifecycle through status transitions. On placement, it calls the Cart-Service to snapshot items, the Payment-Service to process payment, and the Schedule-Service to assign a delivery agent. It supports order cancellation with refund triggering and reorder functionality.

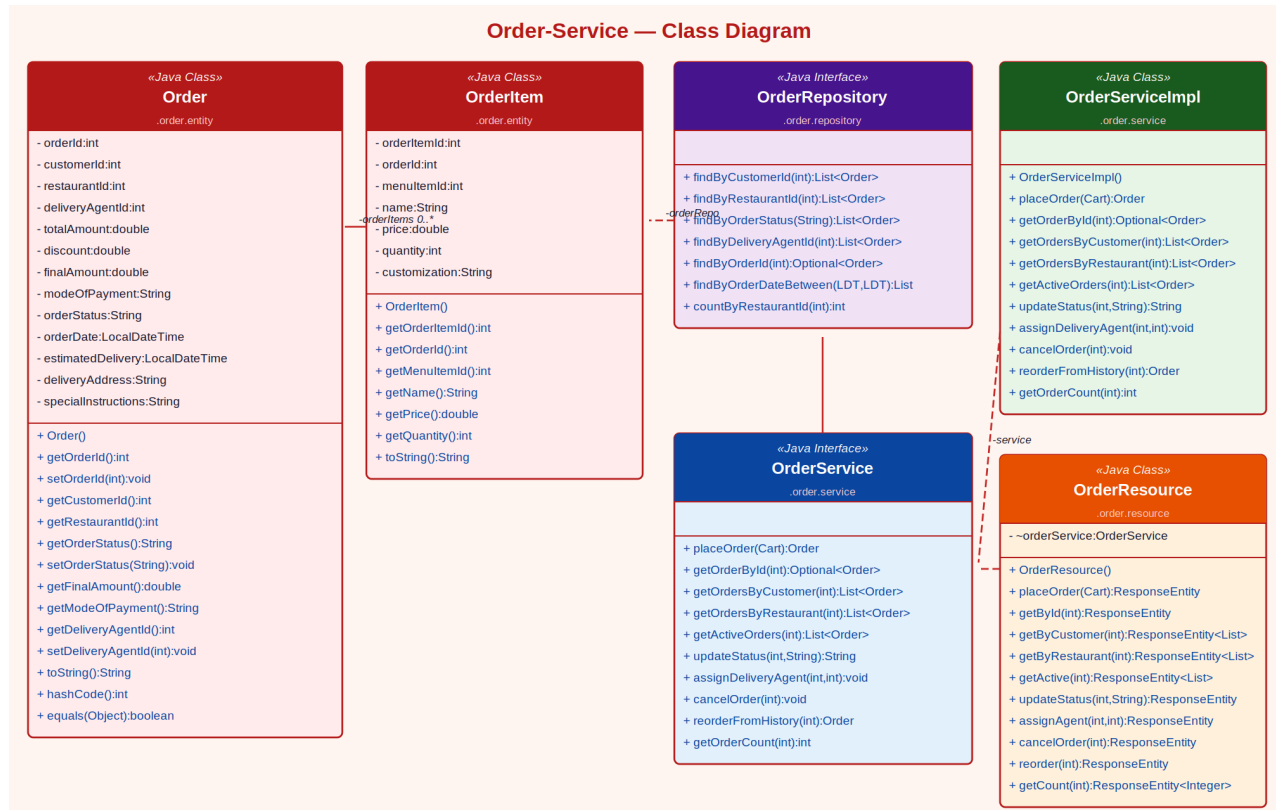


Figure 6: Order-Service — Class Diagram

Component	Type	Key Attributes / Methods
Order	Java Class (Entity)	orderId, customerId, restaurantId, deliveryAgentId, totalAmount, discount, finalAmount, modeOfPayment, orderStatus (PLACED/CONFIRMED/PREPARING/PICKED_UP /DELIVERED/CANCELLED), orderDate, estimatedDelivery, deliveryAddress, specialInstructions
OrderItem	Java Class (Entity)	orderItemId, orderId, menuItemId, name, price, quantity, customization — immutable snapshot of cart at time of order
OrderRepository	Java Interface	findByCustomerId(), findByRestaurantId(), findByOrderStatus(), findByDeliveryAgentId(),

Component	Type	Key Attributes / Methods
		findByOrderId(), findByOrderDateBetween(), countByRestaurantId()
OrderServiceImpl	Java Class	placeOrder(), getOrderById(), getOrdersByCustomer(), getOrdersByRestaurant(), getActiveOrders(), updateStatus(), assignDeliveryAgent(), cancelOrder(), reorderFromHistory(), getOrderCount()
OrderService	Java Interface	Declares all order placement, retrieval, status management, agent assignment, cancellation, and reorder operations
OrderResource	Java Class (REST)	Exposes /orders endpoints: POST (place), GET (by id/customer/restaurant/active), PUT (updateStatus/assignAgent/cancel), POST (reorder), GET count

4.6 Payment / Wallet-Service

The Payment/Wallet-Service processes order payments and manages customer e-wallets. Each Payment record is linked to exactly one order and stores the amount, payment mode, transaction ID, currency, and status. The Wallet entity tracks a customer's running balance with a linked list of WalletStatements recording deposits and debits. The service validates sufficient balance before wallet payments and triggers refunds on order cancellation.

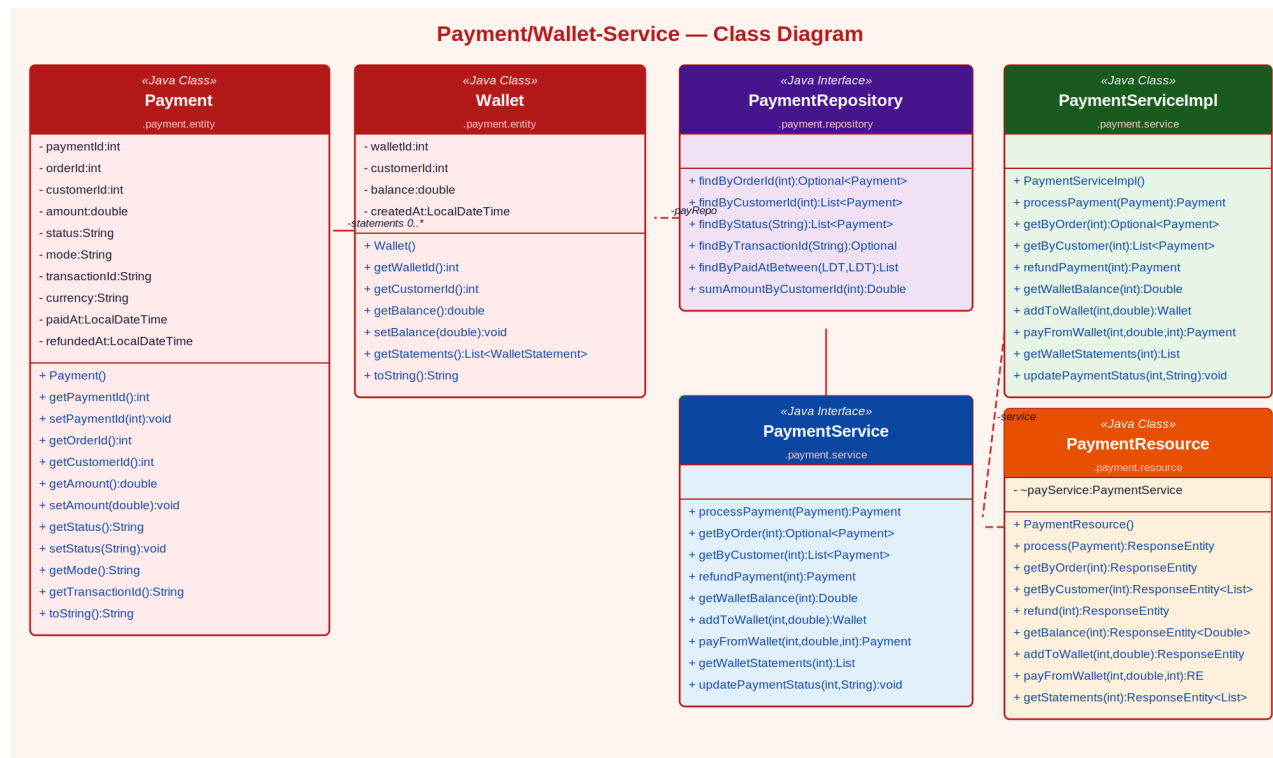


Figure 7: Payment/Wallet-Service — Class Diagram

Component	Type	Key Attributes / Methods
Payment	Java Class (Entity)	paymentId, orderId, customerId, amount, status (PENDING/PAID/REFUNDED/FAILED), mode (CARD/UPI/WALLET/COD), transactionId, currency, paidAt, refundedAt
Wallet	Java Class (Entity)	walletId, customerId, balance; contains 0..* WalletStatements with deposit/debit records
PaymentRepository	Java Interface	findByOrderId(), findByCustomerId(), findByStatus(), findByTransactionId(), findByPaidAtBetween(), sumAmountByCustomerId()
PaymentServiceImpl	Java Class	processPayment(), getByOrder(), getByCustomer(), refundPayment(), getWalletBalance(), addToWallet(),

Component	Type	Key Attributes / Methods
		payFromWallet(), getWalletStatements(), updatePaymentStatus()
PaymentService	Java Interface	Declares all payment processing, refund, wallet top-up, wallet payment, and statement retrieval operations
PaymentResource	Java Class (REST)	Exposes /payments and /wallet endpoints: POST (process/addToWallet/payFromWallet), GET (byOrder/byCustomer/balance/statements), POST (refund)

4.7 Delivery-Agent-Service

The Delivery-Agent-Service manages the profiles and real-time state of delivery agents. Agents are registered with vehicle details and require admin verification before activation. The service exposes a geolocation-based query to find available agents near a restaurant for order assignment. Live location updates (latitude/longitude) are stored at each check-in to enable real-time tracking on the customer's order tracking screen.

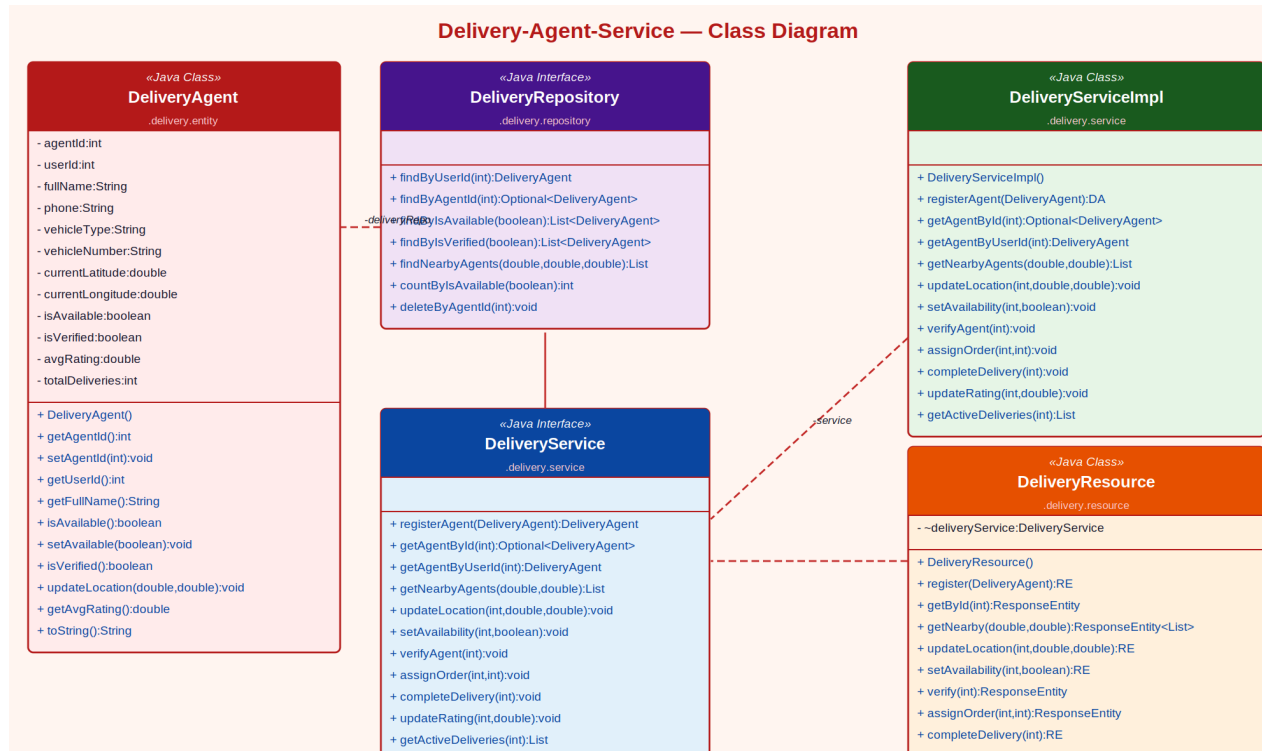


Figure 8: Delivery-Agent-Service — Class Diagram

Component	Type	Key Attributes / Methods
DeliveryAgent	Java Class (Entity)	agentId, userId, fullName, phone, vehicleType, vehicleNumber, currentLatitude, currentLongitude, isAvailable, isVerified, avgRating, totalDeliveries
DeliveryRepository	Java Interface	findByUserId(), findByAgentId(), findByIsAvailable(), findByIsVerified(), findNearbyAgents(), countByIsAvailable(), deleteByAgentId()
DeliveryServiceImpl	Java Class	registerAgent(), getAgentById(), getAgentByUserId(), getNearbyAgents(), updateLocation(), setAvailability(), verifyAgent(), assignOrder(), completeDelivery(), updateRating(), getActiveDeliveries()

Component	Type	Key Attributes / Methods
DeliveryService	Java Interface	Declares agent registration, geo-proximity lookup, location update, availability toggle, verification, assignment, and rating operations
DeliveryResource	Java Class (REST)	Exposes /agents endpoints: POST (register), GET (by id/nearby), PUT (location/availability/verify/rating), POST (assignOrder/completeDelivery), GET (activeDeliveries)

4.8 Review / Rating-Service

The Review/Rating-Service allows customers to submit dual ratings after order completion — a food quality rating for the restaurant and a delivery experience rating for the agent. One review is permitted per order, enforced by a unique constraint on orderId. Average ratings are computed and pushed back to the Restaurant-Service and Delivery-Agent-Service to update their respective avgRating fields, which surface on discovery cards.

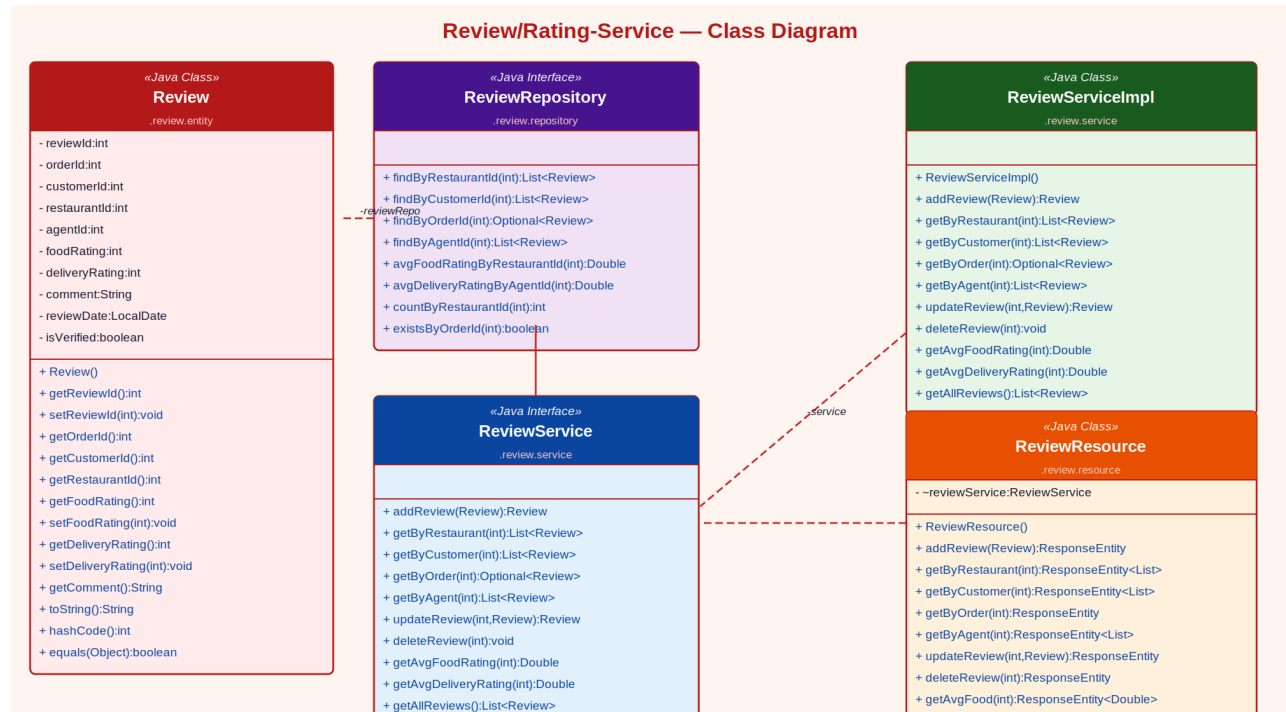


Figure 9: Review/Rating-Service — Class Diagram

Component	Type	Key Attributes / Methods
Review	Java Class (Entity)	reviewId, orderId (unique), customerId, restaurantId, agentId, foodRating (1-5), deliveryRating (1-5), comment, reviewDate, isVerified
ReviewRepository	Java Interface	findByRestaurantId(), findByCustomerId(), findByOrderId(), findByAgentId(), avgFoodRatingByRestaurantId(), avgDeliveryRatingByAgentId(), countByRestaurantId(), existsByOrderId()
ReviewServiceImpl	Java Class	addReview(), getByRestaurant(), getByCustomer(), getByOrder(), getByAgent(), updateReview(), deleteReview(), getAvgFoodRating(), getAvgDeliveryRating(), getAllReviews()

Component	Type	Key Attributes / Methods
ReviewService	Java Interface	Declares dual-rating submission, retrieval by entity, average computation, and moderation operations
ReviewResource	Java Class (REST)	Exposes /reviews endpoints: POST (add), GET (by restaurant/customer/order/agent/all), PUT (update), DELETE, GET (avgFood/avgDelivery)

4.9 Notification-Service

The Notification-Service dispatches multi-channel alerts (in-app, email, SMS) for all order lifecycle events: order placed, confirmed by restaurant, being prepared, picked up by agent, and delivered. It also handles promotional broadcast messages, payment receipt alerts, and restaurant owner new-order pings. The service tracks read/unread state, exposes an unread badge count, and supports bulk dispatch for admin campaigns.

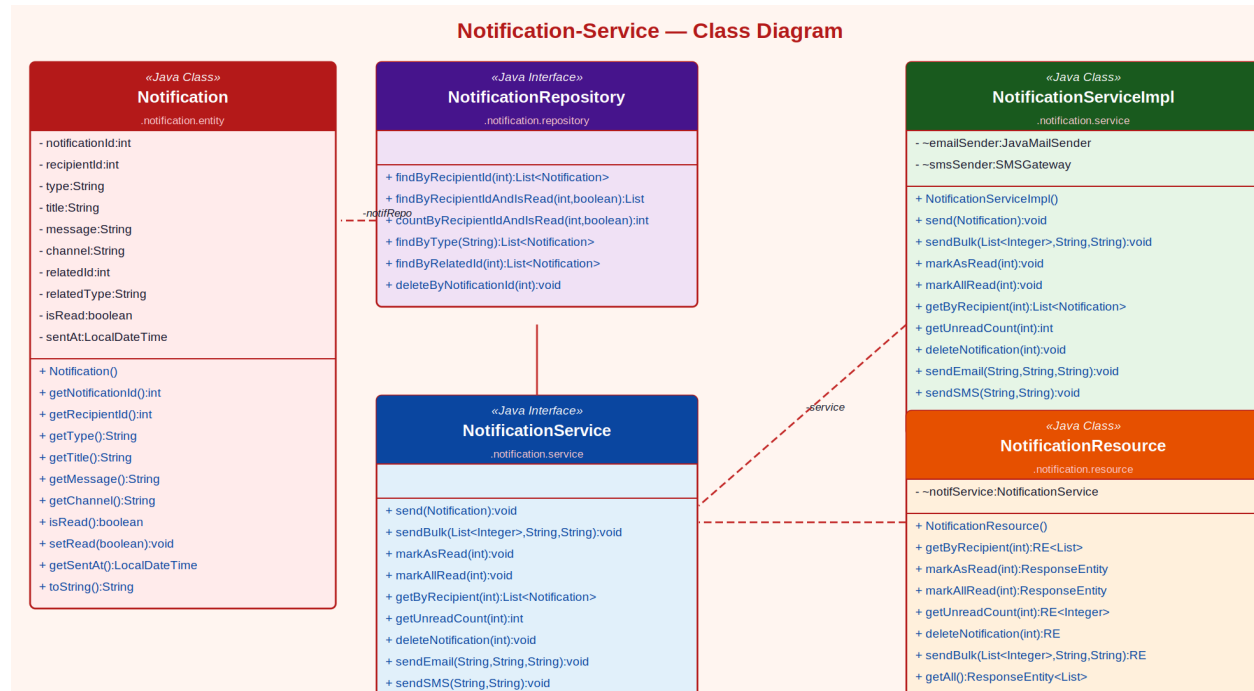


Figure 10: Notification-Service — Class Diagram

Component	Type	Key Attributes / Methods
Notification	Java Class (Entity)	notificationId, recipientId, type (ORDER/PAYMENT/PROMO/DELIVERY), title, message, channel (APP/EMAIL/SMS), relatedId, relatedType, isRead, sentAt
NotificationRepository	Java Interface	findByRecipientId(), findByRecipientIdAndIsRead(), countByRecipientIdAndIsRead(), findByType(), findByRelatedId(), deleteByNotificationId()
NotificationServiceImpl	Java Class	send(), sendBulk(), markAsRead(), markAllRead(), getByRecipient(), getUnreadCount(), deleteNotification(), sendEmail(), sendSMS(), getAll()
NotificationService	Java Interface	Declares notification dispatch (single/bulk/email/SMS), retrieval, read-state management, and deletion operations

Component	Type	Key Attributes / Methods
NotificationResource	Java Class (REST)	Exposes /notifications endpoints: getByRecipient, markAsRead, markAllRead, getUnreadCount, delete, sendBulk, getAll

4.10 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC integration layer that wires all underlying microservices together and renders Thymeleaf views for customers, restaurant owners, and administrators. It contains three controllers — CustomerController, RestaurantController, and AdminController — each injecting relevant service dependencies via Spring `@Autowired` and delegating to REST Resource endpoints via `RestTemplate` for inter-service coordination.

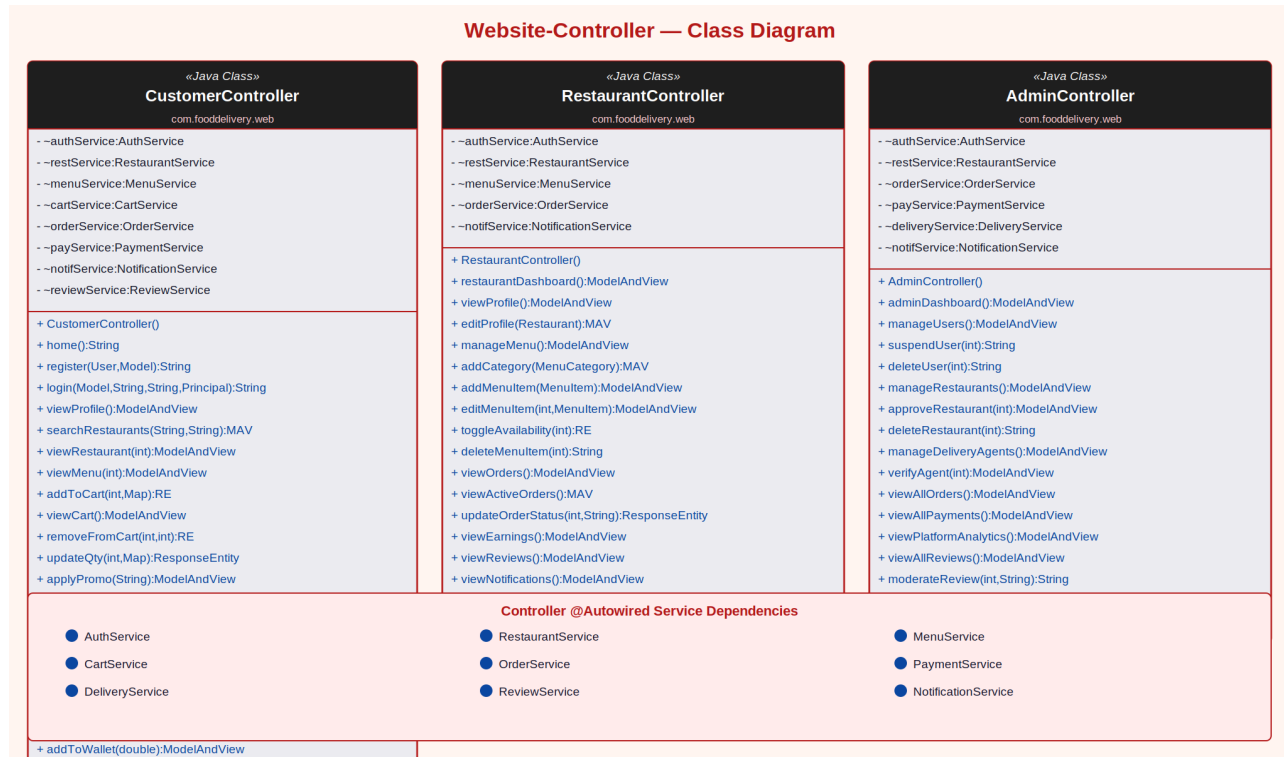


Figure 11: Website-Controller — Class Diagram

Controller	Type	Key Methods
CustomerController	Java Class (MVC)	home(), register(), login(), viewProfile(), searchRestaurants(), viewRestaurant(), viewMenu(), addToCart(), viewCart(), removeFromCart(), updateQty(), applyPromo(), checkout(), placeOrder(), trackOrder(), viewOrderHistory(), cancelOrder(), reorder(), makePayment(), addWallet(), viewWallet(), submitReview(), viewNotifications(), logout()
RestaurantController	Java Class (MVC)	restaurantDashboard(), viewProfile(), editProfile(), manageMenu(), addCategory(), addMenuItem(), editMenuItem(), toggleAvailability(), deleteMenuItem(), viewOrders(), viewActiveOrders(), updateOrderStatus(), viewEarnings(), viewReviews(), viewNotifications()

Controller	Type	Key Methods
AdminController	Java Class (MVC)	adminDashboard(), manageUsers(), suspendUser(), deleteUser(), manageRestaurants(), approveRestaurant(), deleteRestaurant(), manageDeliveryAgents(), verifyAgent(), viewAllOrders(), viewAllPayments(), viewPlatformAnalytics(), viewAllReviews(), moderateReview(), sendPlatformNotification(), generateRevenueReport()

5. Microservices Architecture Overview

QuickBite is structured as a set of independently deployable microservices following the EShoppingZone reference pattern. Each service owns its own database schema, REST API contract, service interface, and business logic. Synchronous inter-service calls use RestTemplate; asynchronous order-event notifications are routed via RabbitMQ.

Microservice	Base Package	Primary Domain
auth-service	com.quickbite.auth	User registration, login, JWT, OAuth2 (Google/GitHub)
restaurant-service	com.quickbite.restaurant	Restaurant profiles, geo-proximity search, approval, open-toggle
menu-service	com.quickbite.menu	Menu categories, items, availability toggle, veg/non-veg filter
cart-service	com.capgemini.cartservice	Shopping cart, CartItem management, promo code, totals
order-service	com.cg.order.orderservice	Order placement, lifecycle status, agent assignment, reorder
payment-service	com.quickbite.payment	Payment processing, wallet balance, deposits, refunds, statements
delivery-service	com.quickbite.delivery	Agent profiles, geo-availability, live location, delivery assignment
review-service	com.quickbite.review	Dual food + delivery ratings, average computation, moderation
notification-service	com.quickbite.notification	In-app, email, SMS alerts; bulk dispatch; read-state tracking
quickbite-web	com.quickbite.web	Spring MVC controllers, Thymeleaf view rendering

6. Non-Functional Requirements

Category	Requirement
Performance	Restaurant discovery and menu load must respond within 1.5 seconds for 50,000 concurrent users; order placement API within 2 seconds
Scalability	Each microservice independently scalable via Docker/Kubernetes; Redis caches restaurant listings and cart sessions for high-frequency reads
Security	Passwords stored as bcrypt hashes; JWT tokens expire in 24 hours; OAuth2 for social login; HTTPS enforced; cart and order APIs gated by JWT
Availability	99.9% uptime SLA; health-check endpoints per service; graceful degradation on notification or review service outage
Data Integrity	Cart-to-order conversion is atomic; wallet debit and payment record creation are transactional to prevent double-charge or negative balance
Real-time Tracking	Delivery agent location updates are pushed to the customer's order tracking page via WebSocket (STOMP) every 15 seconds
Geo-proximity	Nearby restaurant and agent discovery uses the Haversine formula on stored latitude/longitude; configurable radius in kilometres
Audit Trail	All order status transitions, payment events, and review moderation actions are logged with actor, timestamp, and before/after values
Usability	Responsive UI on desktop and mobile; WCAG 2.1 AA accessibility; audio notification for restaurant owner new-order alerts
Maintainability	Services independently deployable; RESTful API contracts versioned under /api/v1/

7. Proposed Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA)
Authentication	JWT (JSON Web Tokens), Spring Security OAuth2, Google / GitHub OAuth
Database	MySQL (primary relational store per service); Redis (restaurant listing cache, cart sessions, agent location cache)
Geo-Proximity Search	MySQL spatial queries with Haversine formula; optionally Elasticsearch for advanced restaurant search
Real-time Location	WebSocket (STOMP) via Spring WebSocket for live delivery agent location broadcast to customers
Frontend	Thymeleaf templates with Bootstrap 5 and Leaflet.js for live map tracking; or React.js SPA
File / Image Storage	AWS S3 for restaurant and menu item image uploads; CloudFront CDN for delivery
Payment Gateway	Razorpay / Stripe for card and UPI payments; internal wallet service for prepaid balance
Notifications	JavaMailSender (SMTP) for email; Twilio or AWS SNS for SMS; WebSocket for in-app real-time alerts
Messaging	RabbitMQ for asynchronous order-event notification dispatch and delivery assignment events
Containerisation	Docker with Docker Compose; Kubernetes for production autoscaling and blue-green deployments
API Documentation	Swagger / OpenAPI 3.0 for all REST endpoints across all services
CI/CD	GitHub Actions or Jenkins pipeline for automated build, test, and deployment

8. Glossary

Term	Definition
Customer	A registered user who browses restaurants, places food orders, and provides reviews on QuickBite
Restaurant Owner	A merchant who registers their restaurant, manages the digital menu, and processes incoming customer orders
Delivery Agent	A registered rider who accepts pickup and delivery assignments and updates live location during delivery
Order Lifecycle	The sequence of states an order passes through: Placed → Confirmed → Preparing → Picked Up → Delivered
Cart	A temporary session storing a customer's selected menu items from a single restaurant prior to checkout
CartItem	A single menu item entry in the cart capturing quantity, price snapshot, and customisation notes
OrderItem	An immutable snapshot of a CartItem saved to the Order on placement for historical accuracy
Wallet	A pre-loaded digital balance linked to a customer account used for cashless order payments
Promo Code	A discount voucher applied at cart level to reduce the final order total
Delivery Radius	The maximum distance in kilometres from a restaurant within which it accepts delivery orders
Geo-proximity	Location-based restaurant and agent discovery using Haversine distance calculation on GPS coordinates
Food Rating	A 1–5 star score given by the customer rating the quality of food from a restaurant
Delivery Rating	A 1–5 star score given by the customer rating the professionalism and speed of the delivery agent
Approval	The admin action required to activate a newly registered restaurant before it becomes visible to customers
Verification	The admin action required to activate a delivery agent after reviewing their identity and vehicle documents
COD	Cash on Delivery — a payment mode where the customer pays cash directly to the delivery agent on receipt